Vehicle Positioning and LoS/NLoS Classification Using Machine Learning and Neural Networks

Ashutosh Kumar
ECE 553/653 Neural Networks — Project 1: Vehicle Positioning

***Abstract*—This report studies two tasks on a measured channel state information (CSI) dataset of an underground parking lot: line-of-sight/non-line-of-sight (LoS/NLoS) classification and estimation of the vehicle position. Three input formats are designed from the 4×1632 complex CSI — a normalized amplitude image, the channel impulse response (CIR) and 46 physical features — and five machine-learning (ML) models are compared with eight neural-network (ANN) architectures in a configuration study of activations, output layers, losses, optimizers, learning rates, batch sizes and input scaling. On the official validation set, whose positions also appear in training, the best models classify all samples correctly and KNN fingerprinting locates the vehicle with a mean error of 0.004 m, while the best regression network reaches 0.232 m. Efficiency, robustness to noisy and lost CSI packets and generalization to unseen positions are also evaluated: noise/packet-loss augmentation keeps the CNN accuracy above 98 % under severe corruption, but all models degrade strongly on positions never seen in training.**

***Index Terms*—channel state information, fingerprinting, LoS/NLoS identification, vehicle positioning, convolutional neural network, machine learning.**

I. Introduction

Accurate positioning is a key service of 5G networks, also for vehicles in places where satellite navigation fails, such as tunnels and underground garages [1]. Fingerprinting methods learn the map between a radio measurement and a position [2]; channel state information (CSI) is a much richer fingerprint than the received power, because it describes the channel on every subcarrier and antenna [3]. Deep networks have been shown to learn such fingerprints directly from CSI [4], [5]. Knowing whether a link is LoS or NLoS is equally important: NLoS links bias range-based positioning, and the classical identification methods use statistics of the impulse response such as the kurtosis, the mean excess delay and the RMS delay spread [6], [7].

Following the project description [8], this work: 1) designs pre-processing methods and input formats for ML and ANN methods; 2) compares five ML models and eight ANN architectures for LoS/NLoS classification and position estimation under many configurations; 3) evaluates training time, latency and model size; and 4) studies robustness on an emulated unreliable network and generalization to new positions. Section II describes the methodology step by step, Section III presents the results and Section IV concludes.

II. Methodology

The pipeline (Fig. 1) has six steps: **1)** read the CSI and labels; **2)** pre-process the CSI into three input formats; **3)** train five ML models and **4)** eight ANNs on the training set; **5)** vary one design choice at a time to select the configurations; **6)** evaluate accuracy, efficiency, robustness and generalization. Each task has its own notebook (*LoS_NLoS_classification.ipynb*, *Position estimation.ipynb*), written in Python with PyTorch [9] and scikit-learn [10].

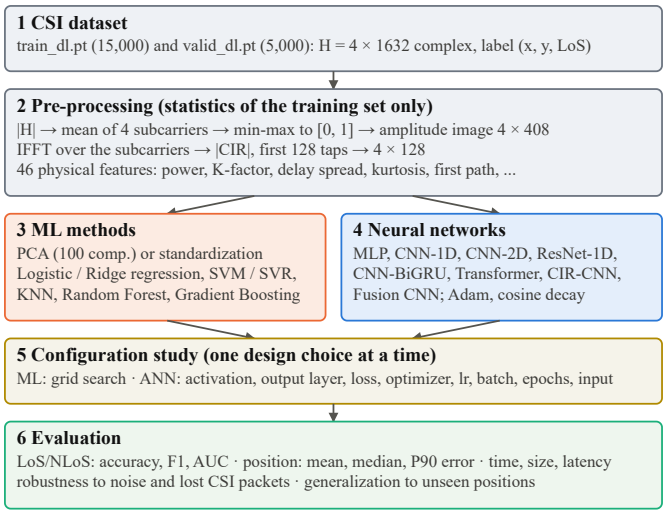


Fig. 1. Methodology used for both tasks (LoS/NLoS classification and position estimation): from the raw CSI to the evaluation.

A. Step 1 — Dataset and evaluation protocol

The CSI was measured at 476 vehicle positions in an underground parking lot [8]. Each sample is a complex 4×1632 matrix (4 RRU antennas, 1632 subcarriers) with the label (x, y, LoS). The training set has 15,000 and the validation set 5,000 samples; both contain *all* 476 positions and 18.7 % of the validation samples are NLoS. All models are trained on *train_dl.pt* and evaluated on *valid_dl.pt*, which is also used to choose the hyper-parameters and the early-stopping epoch. Because the two sets share their positions, this protocol measures the recognition of known fingerprints; a second protocol therefore splits all 20,000 samples by position (380 seen, 96 unseen positions).

Metrics: accuracy, precision, recall and F1 of the NLoS class and ROC-AUC for classification; for positioning the Euclidean error $\|\hat{p}-p\|$, summarized by its mean, median and 90th percentile (P90). Baselines: always "LoS" gives 81.3 % accuracy and always the mean training position a mean error of 13.36 m.

B. Step 2 — Data pre-processing and input formats

Amplitude image. The modulus |H| is used because the raw phase contains random timing and oscillator offsets. Every 4 neighbouring subcarriers are averaged (1632→408), which keeps the frequency response, reduces noise and makes training 4× cheaper. Values are min-max scaled to [0, 1] with the *training* minimum and maximum, $\tilde{x} = (x-x_{min})/(x_{max}-x_{min})$, giving a 4×408 "image" whose rows are the antennas; dB and per-sample scaling are compared in Section III.

Channel impulse response. $|h_a[\tau]| = |\mathrm{IFFT}_k(H_{a,k})|$ describes the channel in the delay domain; the first 128 taps hold 99.4 % of the energy and are kept (4×128). A LoS channel has one dominant early path, whereas NLoS energy is spread over many weaker paths (Fig. 2a).

Physical features. Forty-six features are computed per sample: for each antenna the received power, the standard deviation of |H| in dB over frequency, the Rician K-factor estimated from the moments of $|H|^2$ [11], the kurtosis, the delay and energy ratio of the strongest tap, the mean excess delay, the RMS delay spread and the delay and power of the first path [6], [7]; and the 6 inter-antenna correlations of |H|. Fig. 2 confirms that LoS samples have larger K-factors and smaller delay spreads.

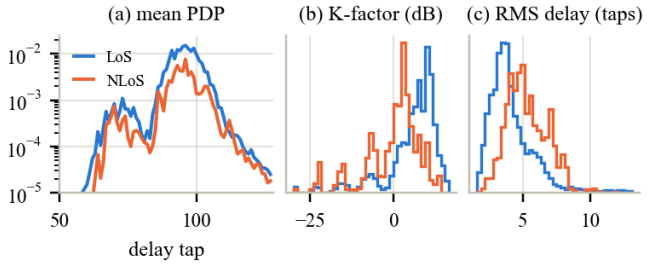


Fig. 2. LoS vs NLoS signatures in the training set: (a) mean power delay profile (PDP); distributions of (b) the Rician K-factor and (c) the RMS delay spread (mean of the 4 antennas).

Model inputs and targets. The ML models receive vectors: the flattened amplitude image or CIR projected on its principal components (PCA [12]; 100 components keep 99.33 % of the amplitude variance), the standardized physical features, or both. The ANNs receive the normalized tensors and learn their own features. For positioning, the network targets are the standardized coordinates $(x-\mu_x)/\sigma_x$ and $(y-\mu_y)/\sigma_y$, so both outputs have the same scale; predictions are mapped back to metres. The received power follows a log-distance path-loss law [13] with exponent n = 1.55, but it explains only 26 % of the variance in the garage: a regression of the log-distance on the power has a mean distance error of 7.9 m (mean-distance baseline 9.1 m), which motivates the use of the full CSI.

C. Step 3 — Machine-learning methods

Table I lists the ML methods, used for both tasks: logistic regression (Ridge linear regression for positioning), the support vector machine/regressor with RBF kernel [14], k-nearest neighbours [15] — the classical fingerprinting method [2] —, Random Forest [16] and histogram gradient boosting [17]. Each model is one pipeline (PCA or scaling → model) fitted on the training set.

TABLE I
DESIGN OF THE ML METHODS (BOTH TASKS)

Model	Input (best)	Decision function / activation	Output
Logistic / linear regression	PCA |H| or poly-2 phys.	$w^Tx + b$; sigmoid (cls), identity (pos)	p(NLoS) / (x, y)
SVM / SVR (RBF)	PCA |H|, standardized	$\Sigma\alpha_i\exp(-\gamma\|x-x_i\|^2)+b$	sign / (x, y)
KNN	PCA |H|	vote / mean of k nearest fingerprints	class / (x, y)
Random Forest	phys. (cls), PCA |H| (pos)	average of bagged trees	class / (x, y)
Gradient Boosting	phys. (cls), PCA |H| (pos)	sum of boosted trees; sigmoid (cls)	p(NLoS) / (x, y)

D. Step 4 — Artificial neural networks

Table II summarizes the eight architectures and Fig. 3 shows the CNN-1D in detail. The 1-D CNN treats the 4 antennas as input channels and slides along frequency, so it learns local patterns such as fading notches; the 2-D CNN mixes the antennas with a (4×3) kernel; ResNet-1D adds skip connections [18]; CNN-BiGRU reads the frequency response as a sequence [19]; the Transformer applies self-attention to 51 patches [20]; the fusion CNN combines an amplitude and a CIR branch. Convolutions are followed by batch normalization [21] and ReLU [22]; dropout 0.2 [23] regularizes the dense head.

TABLE II
ANN ARCHITECTURES (HIDDEN ACTIVATION RELU, BATCHNORM AFTER EVERY CONVOLUTION)

Network	Input	Hidden layers	Param.
MLP	|H| flat 1632	FC 512 → 256, dropout 0.2	968k
CNN-1D	|H| 4×408	Conv k=7,5,3 (32/64/128) + pool, FC 128	168k
CNN-2D	|H| 1×4×408	Conv (3×7),(3×5),(4×3), FC 128	99k
ResNet-1D	|H| 4×408	stem + 4 residual blocks, global pool	226k
CNN-BiGRU	|H| 4×408	2 conv blocks + BiGRU(64) over 102 steps	62k
Transformer	|H| 51 patches	2 encoder layers, d=64, 4 heads	72k
CIR-CNN	CIR 4×128	CNN-1D on the impulse response	168k
Fusion CNN	|H| + CIR	two conv branches, concatenated	162k

Output layer: 2 logits + softmax with cross-entropy (classification); 2 linear neurons on standardized (x, y) with MSE (positioning). Parameters of the classification networks.

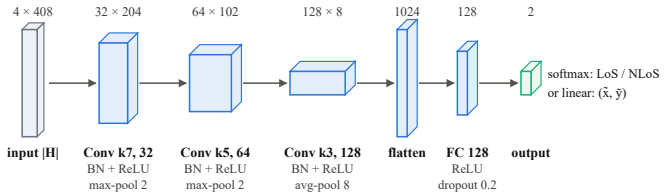


Fig. 3. Architecture of the CNN-1D with the tensor shapes for the amplitude input (BN: batch normalization). The same body is used for both tasks; only the output layer changes.

Output layers and losses. For classification the output layer has 2 neurons with softmax and cross-entropy (the arg-max gives the class). For positioning 2 linear (identity) neurons predict the standardized coordinates with the MSE loss.

E. Step 5 and 6 — Training, configurations and evaluation

All networks use Adam [24], learning rate 10^{-3} with cosine decay [25], batch 64 and *early stopping*: at most 25 epochs for classification and 100 for positioning (200 for the tuned network, 50 in the positioning configuration study); training

stops when the validation cross-entropy (classification) or mean position error (positioning) has not improved for 5 or 15 epochs, and the best checkpoint is restored and saved (*LoS_best_model.pt*, *Position_best_model.pt*). All scaling statistics (min-max, PCA, standardization, target mean and deviation) come from the training data only, which the notebooks verify. The ML models are tuned by grid search; for the CNN-1D one choice is changed at a time: hidden activation (ReLU, LeakyReLU, ELU, GELU, Tanh, Sigmoid), output layer (sigmoid neuron, class-weighted softmax; for positioning a bounded output and a 476-class softmax over the reference positions), loss (L1, Huber [26], Euclidean distance), optimizer, learning rate, batch size, epoch budget and input scaling. Efficiency is measured by the training time, the single-sample CPU latency and the model size.

III. Results

A. LoS/NLoS classification with ML

Table III compares the input formats. Non-linear models reach F1 > 0.99 on the amplitude PCA features, while the linear model reaches only 0.58: LoS/NLoS is not linearly separable in the amplitude space. The 46 physical features are as good as 100 PCA components for the tree ensembles and, with degree-2 polynomial products, raise logistic regression to F1 = 0.979. The grid searches (25 settings for logistic regression, 30 for KNN, 12 each for SVM, Random Forest and boosting) selected polynomial logistic regression (C = 1.0), KNN with k = 1 on 100 components, the SVM with C = 100 and $\gamma = 0.01$, 300 unrestricted trees and boosting with learning rate 0.3. KNN with k = 1 is perfect because every validation sample has near-identical training neighbours from the same position.

Table III
Input formats for ML: NLoS F1 (classification) / mean error in m (positioning)

Model	Amplitude PCA	CIR PCA	Physical (46)	Phys. + CIR
Log./Lin. regression	0.576 / 8.31	0.328 / 8.08	0.585 / 6.48	0.633 / 5.92
SVM (RBF)	0.977 / –	0.847 / –	0.901 / –	0.932 / –
KNN	0.998 / 0.03	0.948 / 1.47	0.973 / 0.88	0.972 / 1.00
Random Forest	0.997 / 0.58	0.950 / 2.30	0.996 / 0.76	0.986 / 1.20
Gradient Boosting	0.999 / 1.21	0.958 / 3.21	0.995 / 1.54	0.994 / 1.73

Default hyper-parameters, official validation set. SVR was tuned on the amplitude features only.

B. LoS/NLoS classification with ANN

Five of the eight networks classify all 5,000 validation samples correctly (Table IV; best epochs 11–25 of at most 25); the weakest is the CIR-CNN (99.80 %). Fig. 5a shows the learning curves: the validation loss falls steeply in the first epochs, and early stopping ends training once it stops improving. The confusion matrices (Fig. 5b–e) are nearly diagonal on known positions, but on unseen positions only 30 % (Random Forest) and 48 % (CIR-CNN) of the NLoS samples are detected. Fig. 4 shows the configuration study. Sigmoid hidden units are the worst activation (98.1 %, best cross-entropy 0.063 vs 0.0001) because they saturate and weaken the gradient flow, and a learning rate of 10^{-4} converges more slowly (99 % after 7 epochs). GELU and per-sample input scaling reach 99 % accuracy after only 3 and 2 epochs (baseline 4); batch 256 trains 5.1× faster but less accurately; a single sigmoid output is as accurate as the softmax but less confident (cross-entropy 0.020 vs 0.0001).

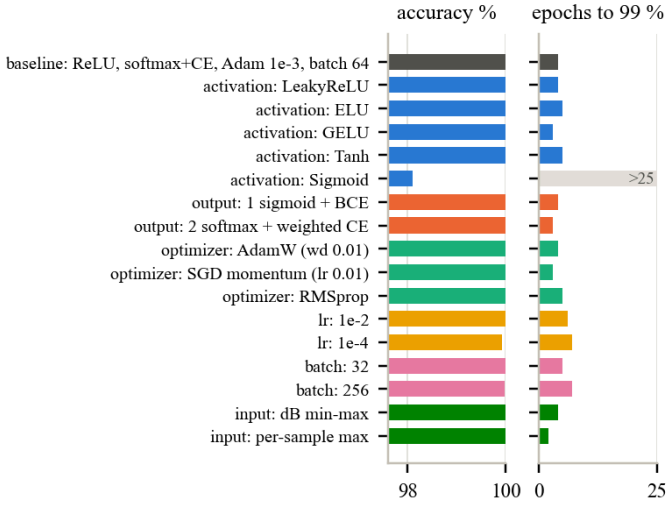


Fig. 4. Configuration study of the CNN-1D for LoS/NLoS classification: one design choice changed at a time (validation accuracy and number of epochs needed to reach 99 % accuracy; grey: not reached).

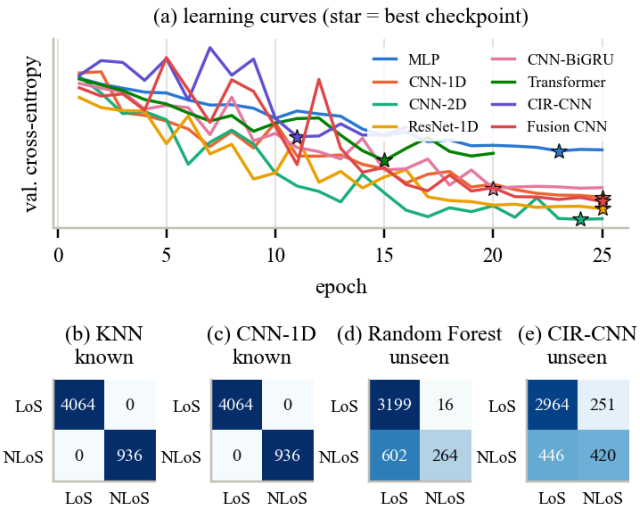


Fig. 5. (a) Learning curves of the eight networks for LoS/NLoS classification (star: best checkpoint restored by early stopping); (b)–(e) confusion matrices of the best ML and ANN models on known (validation) and unseen positions (rows: true class).

C. Position estimation with ML

For positioning the linear model is poor even with many components (Table V), because the map from CSI to the coordinates is strongly non-linear. KNN fingerprinting is almost exact (0.0037 m): with k = 1 it returns the coordinates of the matching training position. Random Forest (150 trees, max_features = 1.0) is the second-best ML model; boosting and the SVR (C = 100.0) interpolate more smoothly but less precisely.

D. Position estimation with ANN

The CNN-BiGRU is the best architecture (Table V); Fig. 7 shows the learning curves: the error keeps falling slowly, so the best checkpoints of 8 of the 10 networks lie in the last 10 % of the cosine schedule; the Transformer and the CIR-CNN reached their best epoch earlier (60, 49). Fig. 6 shows

the configuration study. Output design matters most: treating positioning as a 476-class classification reduces the mean error from 0.530 m to 0.000 m (arg-max) or 0.010 m (probability-weighted mean), because the network then recognizes positions like a fingerprint database. The bounded sigmoid output gives 0.456 m. Among the losses, MSE is the best (MSE 0.530, L1 0.572, Huber 0.546, Euclidean 0.544 m). A longer schedule helps (0.645, 0.530 and 0.460 m for at most 25, 50 and 100 epochs). Sigmoid hidden units are again the worst activation (1.977 m). The tuned network (Tuned CNN-BiGRU, MSE loss, at most 200 epochs, best epoch 189) reaches 0.232 m. A multi-task CNN with one regression and one classification head estimates the position (0.449 m) and the LoS state (100.00 %) with one network.

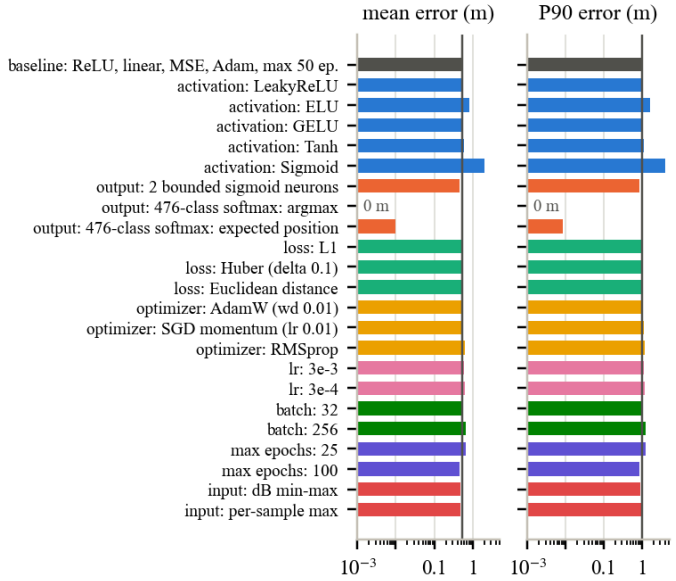


Fig. 6. Configuration study of the CNN-1D for positioning (mean and 90th-percentile error, log scale, 0 m drawn at 1 mm; vertical line: baseline).

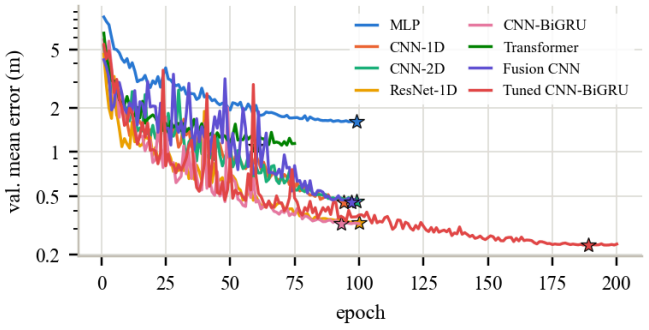


Fig. 7. Learning curves for positioning: validation mean error per epoch (star: best checkpoint restored by early stopping).

E. Efficiency of the algorithms

Training times, latencies and model sizes are listed in Tables IV and V. Most ML models train in a few seconds on the CPU; the slowest is the SVR (RBF) (101 s), and the ANNs need 27–351 s on the GPU. At run time, however, KNN must store the whole fingerprint database (6.9 MB) and compare every query with it, and the largest model is the Random Forest (25 MB). The CNNs need only 0.40–0.94 MB and at most 2.46 ms per sample on one CPU thread; the CNN-BiGRU is the slowest network (up to 8.3 ms) because its GRU steps are sequential.

F. Comparative study of ML and ANN

TABLE IV
LOS/NLOS CLASSIFICATION: FINAL MODELS (VALIDATION SET = KNOWN POSITIONS)

Model	Acc. %	F1	AUC	Ep.	Train s	Lat. ms	MB	Unseen %
Logistic Regression	99.22	0.979	0.9987	–	4.0	0.51	0.04	76.1
SVM (RBF)	99.98	0.999	1.0000	–	2.6	0.35	1.10	76.5
KNN	100.00	1.000	1.0000	–	1.4	1.75	6.78	76.5
Random Forest	99.86	0.996	1.0000	–	1.9	6.16	14.03	84.9
Gradient Boosting	99.92	0.998	0.9998	–	0.6	1.76	0.34	82.6
MLP	99.98	0.999	1.0000	23	32.6	0.31	3.87	78.4
CNN-1D	100.00	1.000	1.0000	25	59.0	1.24	0.68	72.1
CNN-2D	100.00	1.000	1.0000	24	55.4	2.46	0.40	80.5
ResNet-1D	100.00	1.000	1.0000	25	85.5	1.32	0.94	81.8
CNN-BiGRU	100.00	1.000	1.0000	20	42.1	7.21	0.25	–
Transformer	99.98	0.999	1.0000	15	66.5	1.03	0.30	–
CIR-CNN	99.80	0.995	0.9999	11	27.0	0.66	0.68	82.9
Fusion CNN	100.00	1.000	1.0000	25	58.0	1.70	0.66	79.9

Ep.: best (early-stopping) epoch. Train: ML on CPU, ANN on GPU until early stopping. Lat.: CPU latency for one CSI sample. Unseen: accuracy on 96 positions never seen in training (always "LoS" = 78.8 %).

TABLE V
POSITION ESTIMATION: FINAL MODELS (ERRORS IN METRES)

Model	Mean	Median	P90	Ep.	Train s	Lat. ms	MB	Unseen
Linear Regression	3.324	2.859	5.96	–	0.8	0.90	0.04	7.63
KNN	0.004	0.000	0.00	–	1.3	1.29	6.90	11.04
SVR (RBF)	1.198	0.527	3.10	–	101.5	4.86	9.67	11.36
Random Forest	0.469	0.149	1.10	–	31.7	8.90	25.46	9.76
Gradient Boosting	0.951	0.675	1.88	–	7.6	29.07	3.22	9.51
MLP	1.606	1.369	3.04	99	74.6	0.26	3.87	9.52
CNN-1D	0.453	0.351	0.87	94	177.7	0.78	0.68	7.29
CNN-2D	0.463	0.363	0.90	99	158.6	2.10	0.40	6.49
ResNet-1D	0.330	0.253	0.66	100	309.4	2.30	0.94	5.46
CNN-BiGRU	0.324	0.258	0.63	93	188.5	7.58	0.25	–
Transformer	1.086	0.976	1.94	60	221.6	1.38	0.30	–
CIR-CNN	1.277	1.066	2.40	49	66.2	0.21	0.68	6.11
Fusion CNN	0.456	0.372	0.86	97	141.2	0.45	0.66	5.79
Tuned CNN-BiGRU	0.232	0.178	0.45	189	351.0	8.25	0.25	–
Multi-task CNN	0.449	0.351	0.86	100	171.5	1.11	0.68	–

Ep.: best (early-stopping) epoch. Unseen: mean error on 96 positions never seen in training (mean seen position: 13.90 m; nearest seen position: 1.18 m on average).

Accuracy on known positions. For LoS/NLoS the best ML models (KNN, SVM) and the CNNs are equally perfect. For positioning, KNN fingerprinting (0.0037 m) beats every regression network (Tuned CNN-BiGRU: 0.232 m) because the task is effectively a look-up; the networks come close only when their output is a classification over the reference positions (Fig. 6). Fig. 8 shows the full error distributions. *Design effort.* The ML models need hand-made features (PCA, physical features), and their accuracy depends strongly on them (Table III); the networks learn from the normalized CSI but need more design choices and a GPU for fast training. *Cost.* Parametric models (linear, boosting, CNNs) are small and fast at inference; KNN and large forests grow with the data.

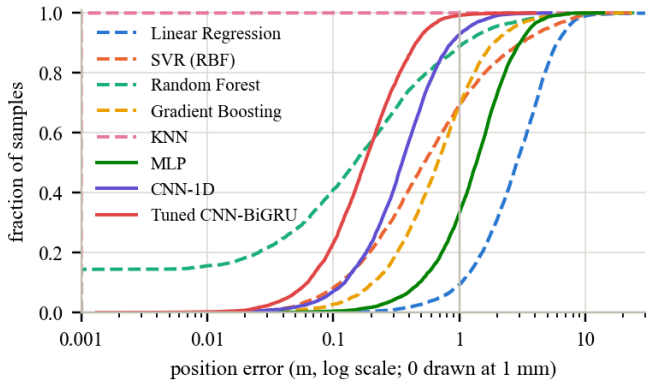


Fig. 8. CDF of the position error on the validation set (dashed: ML, solid: ANN).

G. Robustness on an emulated unreliable network

The validation CSI is corrupted, without retraining, by complex white noise (SNR 30–0 dB) or by lost packets (16 packets of 102 subcarriers, each lost with probability p and zero-filled); all inputs are then recomputed. Two defences are tested: a *receiver-side repair* that removes the phase slope of the strongest path and linearly interpolates the missing subcarriers, and a CNN-1D trained with on-the-fly noise and packet-loss augmentation. Fig. 9 and Table VI show that the clean-trained CNNs and tree ensembles degrade quickly (the CNN-1D falls to 82.0 % at 0 dB and 57.6 % at 50 % loss), whereas KNN is robust to noise down to 5 dB. The repair restores most of the accuracy of every model, and augmentation makes the CNN the most robust classifier (99.0 % at 0 dB, 98.4 % at 50 % loss). The positioning models show the same pattern (Table VI).

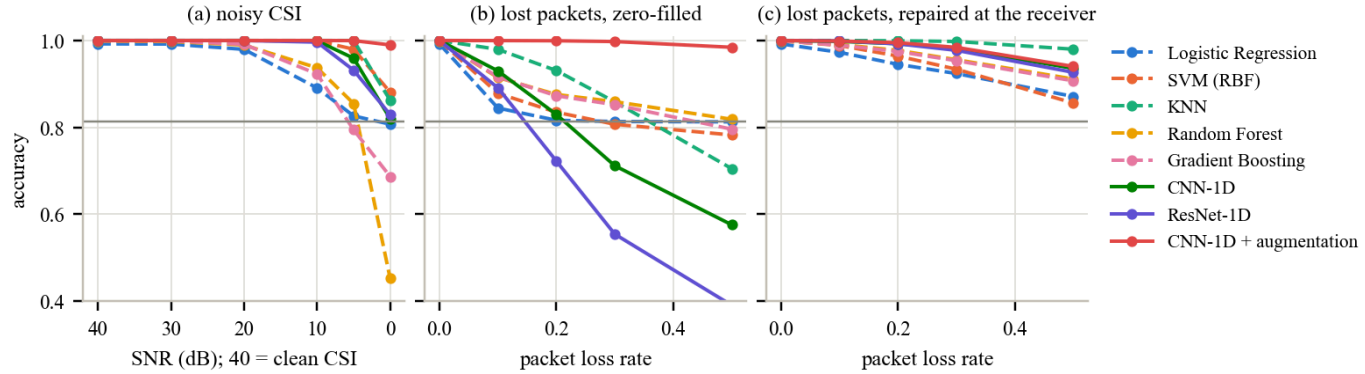


Fig. 9. LoS/NLoS accuracy on corrupted CSI: (a) white noise, (b) lost packets zero-filled, (c) lost packets repaired at the receiver. Grey line: always "LoS".

TABLE VI
EMULATED UNRELIABLE NETWORK: ACCURACY (%) AND MEAN POSITION ERROR (M), NO RETRAINING

Model	10 dB	0 dB	30 %	30 % R	10 dB	0 dB	30 %	30 % R
Log./Lin. regression	89.1	80.7	81.4	92.4	9.59	58.32	2034.24	5.80
SVM / SVR	99.9	88.1	80.7	93.4	2.17	7.92	10.30	4.17
KNN	100.0	86.2	85.9	99.8	0.00	5.67	9.46	0.08
Random Forest	93.7	45.2	85.9	95.6	1.98	8.18	9.81	4.35
CNN-1D	99.9	82.0	71.2	98.2	2.87	11.71	9.24	2.31
ResNet-1D	99.6	82.9	55.4	97.7	3.18	11.23	7.47	2.66
Fusion CNN	100.0	89.6	74.7	98.6	2.66	11.10	7.80	1.95
CNN-1D + augmentation	100.0	99.0	99.8	98.4	0.92	3.05	1.40	2.09

Columns 2–5: LoS/NLoS accuracy; columns 6–9: positioning mean error. dB: SNR of the added noise; 30 %: packet-loss rate (zero-filled); R: lost packets repaired at the receiver.

H. Generalization to unseen positions

For this test all scaling statistics are recomputed from the 380 seen positions, the networks stop early on a held-out 15 % of the seen positions, and the 96 unseen positions are used only once for testing. All models degrade sharply (last columns of Tables IV and V). For LoS/NLoS the best accuracy is 84.9 % (Random Forest) against 78.8 % for always "LoS", and the best NLoS F1 is only 0.55 (CIR-CNN). For positioning the best mean error is 5.46 m (ResNet-1D), better than KNN (11.04 m), but far from the 1.18 m spacing of the reference grid (Fig. 10). The amplitude fingerprint is highly position-specific: with about 1 m between positions the models memorize positions rather than learn the geometry, so the validation scores of Tables IV and V measure fingerprint recognition. Denser measurements, phase-difference or angle features and physics-guided models are needed for generalization.

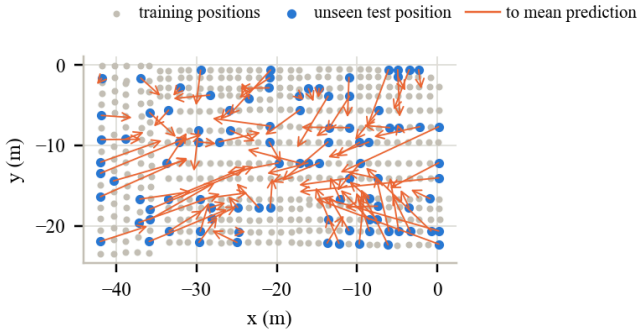


Fig. 10. ResNet-1D on unseen positions: arrow from the true position to the mean predicted position.

I. Screenshots of successful runs

Fig. 11 shows the final result cells of both notebooks after a complete run.

```
In [49]: final_table = summary[['type', 'accuracy', 'F1', 'AUC', 'best_epoch', 'train_s', 'cpu_latency_ms', 's
             unseen_table[['accuracy']].rename(columns={'accuracy': 'unseen_acc'})).round(4)
         final_table
```

Out[49]:

model	type	accuracy	F1	AUC	best_epoch	train_s	cpu_latency_ms	size_MB	unseen_acc
Logistic Regression	ML	0.9922	0.9791	0.9987	<NA>	3.9756	0.5084	0.0387	0.7613
SVM (RBF)	ML	0.9998	0.9995	1.0000	<NA>	2.6224	0.3494	1.0957	0.7650
KNN	ML	1.0000	1.0000	1.0000	<NA>	1.3918	1.7482	6.7819	0.7650
Random Forest	ML	0.9986	0.9962	1.0000	<NA>	1.9363	6.1562	14.0342	0.8486
Gradient Boosting	ML	0.9992	0.9979	0.9998	<NA>	0.5664	1.7551	0.3365	0.8260
MLP	ANN	0.9998	0.9995	1.0000	23	32.6246	0.3125	3.8746	0.7841
CNN-1D	ANN	1.0000	1.0000	1.0000	25	59.0470	1.2437	0.6810	0.7207
CNN-2D	ANN	1.0000	1.0000	1.0000	24	55.4111	2.4601	0.4044	0.8047
ResNet-1D	ANN	1.0000	1.0000	1.0000	25	85.5492	1.3186	0.9382	0.8179
CNN-BiGRU	ANN	1.0000	1.0000	1.0000	20	42.0722	7.2106	0.2535	NaN
Transformer	ANN	0.9998	0.9995	1.0000	15	66.4673	1.0340	0.3001	NaN
CIR-CNN	ANN	0.9980	0.9947	0.9999	11	27.0335	0.6640	0.6810	0.8292
Fusion CNN	ANN	1.0000	1.0000	1.0000	25	57.9725	1.7001	0.6644	0.7988

Fig. 11a. Final result table of *LoS_NLoS_classification.ipynb*.

```
In [49]: final_table = summary[['type', 'mean_m', 'median_m', 'p90_m', 'best_epoch', 'train_s', 'cpu_latency_ms', 'size_M
             unseen_table[['mean_m']].rename(columns={'mean_m': 'unseen_mean_m'})).round(3)
         final_table
```

Out[49]:

model	type	mean_m	median_m	p90_m	best_epoch	train_s	cpu_latency_ms	size_MB	unseen_mean_m
Linear Regression	ML	3.324	2.859	5.965	<NA>	0.805	0.900	0.039	7.632
KNN	ML	0.004	0.000	0.000	<NA>	1.330	1.289	6.902	11.044
SVR (RBF)	ML	1.198	0.527	3.096	<NA>	101.490	4.861	9.666	11.360
Random Forest	ML	0.469	0.149	1.097	<NA>	31.652	8.904	25.458	9.756
Gradient Boosting	ML	0.951	0.675	1.883	<NA>	7.573	29.070	3.219	9.511
MLP	ANN	1.606	1.370	3.038	99	74.565	0.261	3.875	9.517
CNN-1D	ANN	0.453	0.352	0.866	94	177.724	0.780	0.681	7.289
CNN-2D	ANN	0.463	0.363	0.896	99	158.564	2.102	0.404	6.485
ResNet-1D	ANN	0.330	0.253	0.658	100	309.413	2.302	0.938	5.460
CNN-BiGRU	ANN	0.324	0.258	0.625	93	188.467	7.580	0.253	NaN
Transformer	ANN	1.086	0.976	1.943	60	221.614	1.376	0.300	NaN
CIR-CNN	ANN	1.277	1.066	2.396	49	66.229	0.208	0.681	6.107
Fusion CNN	ANN	0.456	0.372	0.863	97	141.232	0.450	0.664	5.785
Tuned CNN-BiGRU	ANN	0.232	0.178	0.453	189	350.985	8.252	0.253	NaN
Multi-task CNN-1D	ANN	0.449	0.351	0.857	100	171.483	1.110	0.682	NaN

Fig. 11b. Final result table of *Position estimation.ipynb* (errors in metres).

IV. Conclusion

Three input formats (normalized amplitude, CIR, physical features) were designed, and five ML models were compared with eight ANNs in more than 100 configurations. On known positions LoS/NLoS classification is solved by both families, and positioning is solved best by KNN fingerprinting; among the regression methods the networks with convolutional front-ends are the most accurate (best: Tuned CNN-BiGRU, 0.232 m) with models of at most 0.94 MB. The output layer and the hidden activation (no sigmoid) matter most; the other design choices change the error much less. Noise and packet-loss augmentation and receiver-side repair make the models robust to an unreliable network, while generalization to unseen positions remains the main open problem.

Limitations. 1) The official validation set shares all positions with the training set and is also used for hyper-parameter selection and early stopping, so its scores are optimistic; the unseen-position split is a fairer test but it is a single split with one random seed and 96 positions, without confidence intervals. 2) Training times compare a CPU (ML) with a laptop GPU (ANN) and vary between runs. 3) Only the CSI amplitude and the CIR magnitude are used; phase information and the antenna geometry are not exploited. 4) Noise and packet loss are simulated (white noise, independent packet losses), not measured on a real network. 5) The ANN positioning results reflect a limited training budget: the best checkpoints of 8 of the 10 networks lie at the end of their schedule, and raising the CNN-1D budget from 50 to 100 epochs lowered its error from 0.530 to 0.460 m, so longer training would probably reduce the errors further.

References

[1] 3GPP, "Study on NR positioning support," 3rd Generation Partnership Project, Sophia Antipolis, France, Tech. Rep. TR 38.855, V16.0.0, Mar. 2019.

[2] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proc. IEEE INFOCOM*, Tel Aviv, Israel, 2000, vol. 2, pp. 775–784.

[3] Z. Yang, Z. Zhou, and Y. Liu, "From RSSI to CSI: Indoor localization via channel response," *ACM Comput. Surv.*, vol. 46, no. 2, pp. 25:1–25:32, Nov. 2013.

[4] X. Wang, L. Gao, S. Mao, and S. Pandey, "CSI-based fingerprinting for indoor localization: A deep learning approach," *IEEE Trans. Veh. Technol.*, vol. 66, no. 1, pp. 763–776, Jan. 2017.

[5] J. Vieira, E. Leitinger, M. Sarajlic, X. Li, and F. Tufvesson, "Deep convolutional neural networks for massive MIMO fingerprint-based positioning," in *Proc. IEEE Int. Symp. Pers., Indoor, Mobile Radio Commun. (PIMRC)*, Montreal, QC, Canada, 2017, pp. 1–6.

[6] I. Güvenç, C.-C. Chong, and F. Watanabe, "NLOS identification and mitigation for UWB localization systems," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Hong Kong, 2007, pp. 1571–1576.

[7] S. Marano, W. M. Gifford, H. Wymeersch, and M. Z. Win, "NLOS identification and mitigation for localization based on UWB experimental data," *IEEE J. Sel. Areas Commun.*, vol. 28, no. 7, pp. 1026–1035, Sep. 2010.

[8] "Project 1: Vehicle positioning," ECE 553/653 Neural Networks, course project description and CSI dataset (*train_dl.pt*, *valid_dl.pt*).

[9] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] C. Tepedelenlioğlu, A. Abdi, and G. B. Giannakis, "The Ricean K factor: Estimation and performance analysis," *IEEE Trans. Wireless Commun.*, vol. 2, no. 4, pp. 799–810, Jul. 2003.

[12] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002.

[13] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[14] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[15] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.

[16] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[19] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.

[20] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017, pp. 5998–6008.

[21] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 448–456.

[22] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, Haifa, Israel, 2010, pp. 807–814.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, 2015.

[25] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, 2017.

[26] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Statist.*, vol. 35, no. 1, pp. 73–101, 1964.